```
import sys
import tensorflow as tf

print("Python:", sys.executable)
print("TensorFlow:", tf.__version__)
print("GPU devices:", tf.config.list_physical_devices("GPU"))
```

```
Python: C:\tf-directml\Scripts\python.exe
TensorFlow: 2.10.0
GPU devices: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]
```

```
import tensorflow as tf
import time

print("TensorFlow:", tf.__version__)

for i in range(2):
    device = f"/GPU:{i}"

    try:
        start = time.time()

        with tf.device(device):
            a = tf.random.normal((2048, 2048))
            b = tf.matmul(a, a)

            # Force computation to finish
            result = b.numpy()

        print(f"\n{device}: SUCCESS")
        print("Result shape:", result.shape)
        print("Actual device:", b.device)
        print("Time taken:", round(time.time() - start, 2), "seconds")

    except Exception as e:
        print(f"\n{device}: FAILED")
        print(str(e)[:1000])
```

```
TensorFlow: 2.10.0

/GPU:0: SUCCESS
Result shape: (2048, 2048)
Actual device: /job:localhost/replica:0/task:0/device:GPU:0
Time taken: 0.62 seconds

/GPU:1: SUCCESS
Result shape: (2048, 2048)
Actual device: /job:localhost/replica:0/task:0/device:GPU:1
Time taken: 0.64 seconds
```

Start coding or generate with AI.

```
import os
import platform
import sys

print("Operating system:", platform.platform())
print("Python executable:", sys.executable)
print("Python version:", sys.version)
print("Current working directory:", os.getcwd())
```

```
Operating system: Windows-10-10.0.26300-SP0
Python executable: C:\tf-directml\Scripts\python.exe
Python version: 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
Current working directory: C:\Users\Akhilesh
```

```
!pip install pandas matplotlib tqdm scikit-learn

import os
import shutil
import zipfile
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tqdm.auto import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

ZIP_PATH = Path(r"C:\Users\Akhilesh\Downloads\skin_data.zip")

EXTRACT_TO = Path(r"C:\Users\Akhilesh\Datasets\skin_data")

IMAGE_SIZE = (224, 224)
BATCH_SIZE = 16
SEED = 42

IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

EXTRACT_TO.parent.mkdir(parents=True, exist_ok=True)

print("ZIP path:", ZIP_PATH)
print("Extraction folder:", EXTRACT_TO)
print("TensorFlow version:", tf.__version__)
```

```
Requirement already satisfied: pandas in c:\tf-directml\lib\site-packages (2.3.3)
Requirement already satisfied: matplotlib in c:\tf-directml\lib\site-packages (3.10.9)
Requirement already satisfied: tqdm in c:\tf-directml\lib\site-packages (4.70.1)
Requirement already satisfied: scikit-learn in c:\tf-directml\lib\site-packages (1.7.2)
Requirement already satisfied: numpy>=1.22.4 in c:\tf-directml\lib\site-packages (from pandas) (1.23.5)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in c:\tf-directml\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\tf-directml\lib\site-packages (from pandas) (2026.5)
Requirement already satisfied: tzdata>=2022.7 in c:\tf-directml\lib\site-packages (from pandas) (2026.5)
Requirement already satisfied: contourpy>=1.0.1 in c:\tf-directml\lib\site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\tf-directml\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\tf-directml\lib\site-packages (from matplotlib) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\tf-directml\lib\site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in c:\tf-directml\lib\site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in c:\tf-directml\lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in c:\tf-directml\lib\site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: colorama in c:\tf-directml\lib\site-packages (from tqdm) (0.4.6)
Requirement already satisfied: scipy>=1.8.0 in c:\tf-directml\lib\site-packages (from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in c:\tf-directml\lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\tf-directml\lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in c:\tf-directml\lib\site-packages (from joblib>=1.2.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in c:\tf-directml\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
ZIP path: C:\Users\Akhilesh\Downloads\skin_data.zip
Extraction folder: C:\Users\Akhilesh\Datasets\skin_data
TensorFlow version: 2.10.0
```

```
MARKER = EXTRACT_TO / ".extraction_complete"

if not ZIP_PATH.is_file():
    raise FileNotFoundError(
        f"ZIP file not found: {ZIP_PATH}\n"
        "Check ZIP_PATH and make sure the file is downloaded."
    )

if MARKER.exists():
    print("Dataset extraction was previously completed.")

else:
    zip_size = ZIP_PATH.stat().st_size

    with zipfile.ZipFile(ZIP_PATH, "r") as zf:
        entries = zf.infolist()
        uncompressed_size = sum(item.file_size for item in entries)
        file_count = sum(not item.is_dir() for item in entries)

        free_bytes = shutil.disk_usage(EXTRACT_TO.parent).free

        print(f"ZIP size: {zip_size / 1024**3:.2f} GB")
        print(f"Extracted size: {uncompressed_size / 1024**3:.2f} GB")
        print(f"Files in ZIP: {file_count:,}")
        print(f"Free disk space: {free_bytes / 1024**3:.2f} GB")

        if free_bytes < uncompressed_size + int(1.5 * 1024**3):
            raise RuntimeError(
                "Not enough free disk space. Free up space or "
                "choose a drive with more available storage."
            )

        print("Extracting dataset. Please wait...")
        start = time.time()

        zf.extractall(EXTRACT_TO)
```

```
    MARKER.write_text("Extraction completed successfully.")

    print(f"Extraction completed in {(time.time() - start) / 60:.2f} minutes.")

print("Extraction folder:", EXTRACT_TO)
print("Top-level contents:", [p.name for p in EXTRACT_TO.iterdir()][:20])
```

```
Dataset extraction was previously completed.
Extraction folder: C:\Users\Akhilesh\Datasets\skin_data
Top-level contents: ['.extraction_complete', 'skin data']
```

```
def find_class_root(base_path):
    all_dirs = []
    child_dirs = {}
    image_counts = {}

    for root, dirs, files in os.walk(base_path):
        current = Path(root)
        all_dirs.append(current)
        child_dirs[current] = [current / name for name in dirs]

        image_counts[current] = sum(
            1 for name in files
            if Path(name).suffix.lower() in IMAGE_EXTENSIONS
        )

    for current in sorted(
        all_dirs,
        key=lambda path: len(path.parts),
        reverse=True
    ):
        if current != base_path:
            image_counts[current.parent] = (
                image_counts.get(current.parent, 0)
                + image_counts[current]
            )

    candidates = []

    for current in all_dirs:
        class_count = sum(
            image_counts.get(child, 0) > 0
            for child in child_dirs[current]
        )

        if class_count >= 2:
            candidates.append((class_count, len(current.parts), current))

    if not candidates:
        raise ValueError(
            "Could not find a folder containing at least two "
            "disease-class folders with images."
        )

    return max(candidates, key=lambda item: (item[0], item[1]))[2]
```

```
dataset_path = find_class_root(EXTRACT_TO)

class_names = sorted([
    folder.name
    for folder in dataset_path.iterdir()
    if folder.is_dir()
    and any(
        file.is_file()
        and file.suffix.lower() in IMAGE_EXTENSIONS
        for file in folder.rglob("*")
    )
])

print("Dataset folder:", dataset_path)
print("Number of classes:", len(class_names))
print("Classes:", class_names)

if len(class_names) < 2:
    raise ValueError("At least two disease-class folders are required.")
```

```
Dataset folder: C:\Users\Akhilesh\Datasets\skin_data\skin data
Number of classes: 10
Classes: ['1. Eczema 1677', '10. Warts Molluscum and other Viral Infections - 2103', '2. Melanoma 15.75k', '3. Atopic Dermatitis - 1.25k', '4. Basal Cell Carcinoma (BCC) 3323', '5.
```

```
empty_files = []
file_paths = []
labels = []

print("Checking for empty image files...")

for class_index, class_name in enumerate(class_names):
    class_folder = dataset_path / class_name

    for file in class_folder.rglob("*"):
        if file.is_file() and file.suffix.lower() in IMAGE_EXTENSIONS:
            if file.stat().st_size == 0:
                empty_files.append(file)
            else:
                file_paths.append(str(file))
                labels.append(class_index)

for file in empty_files:
    file.unlink()

print("Empty files removed:", len(empty_files))
print("Total usable image files:", len(file_paths))

class_counts = pd.Series(
    [class_names[label] for label in labels]
).value_counts().reindex(class_names, fill_value=0)

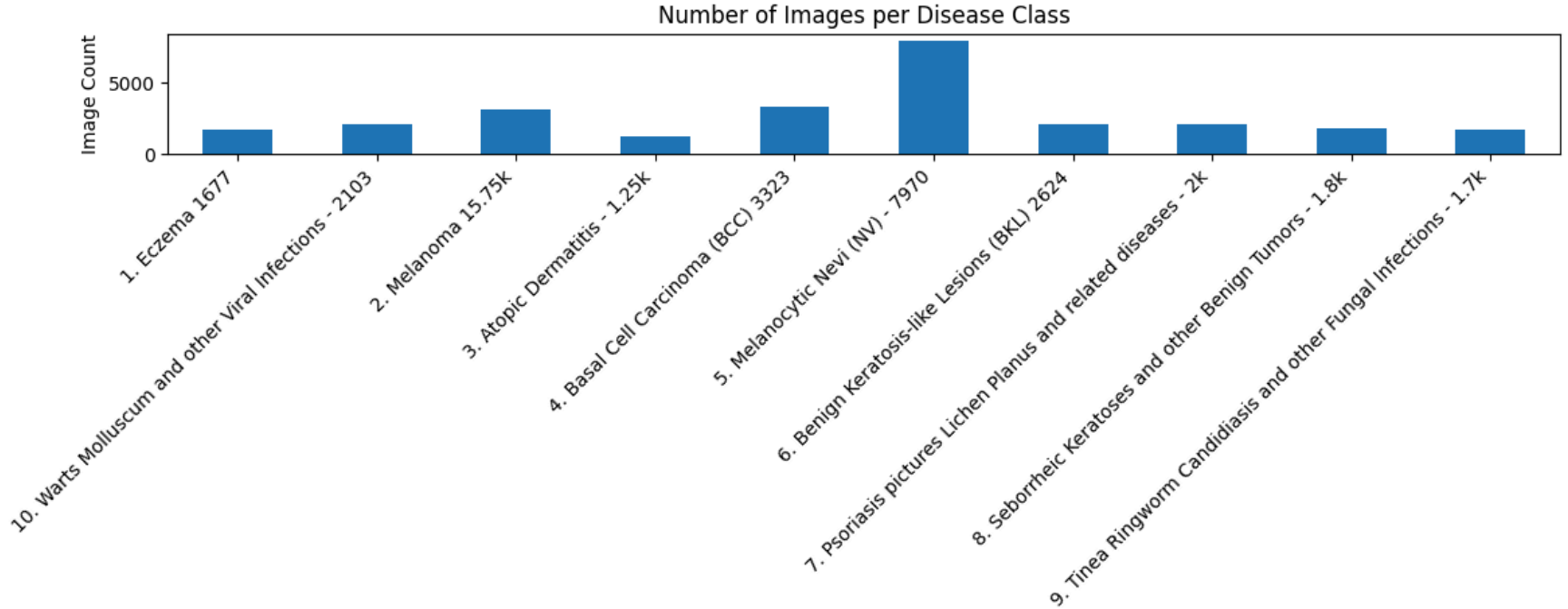
print("\nImages per class:")
print(class_counts.to_string())
```

```
plt.figure(figsize=(12, 5))
class_counts.plot(kind="bar")
plt.title("Number of Images per Disease Class")
plt.xlabel("Disease Class")
plt.ylabel("Image Count")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

if len(file_paths) == 0:
    raise ValueError("No usable images were found.")
```

```
Checking for empty image files...
Empty files removed: 0
Total usable image files: 27153

Images per class:
1. Eczema 1677                                                1677
10. Warts Molluscum and other Viral Infections - 2103         2103
2. Melanoma 15.75k                                            3140
3. Atopic Dermatitis - 1.25k                                  1257
4. Basal Cell Carcinoma (BCC) 3323                            3323
5. Melanocytic Nevi (NV) - 7970                               7970
6. Benign Keratosis-like Lesions (BKL) 2624                   2079
7. Psoriasis pictures Lichen Planus and related diseases - 2k 2055
8. Seborrheic Keratoses and other Benign Tumors - 1.8k        1847
9. Tinea Ringworm Candidiasis and other Fungal Infections - 1.7k 1702
```




```
gpus = tf.config.list_physical_devices("GPU")

print("TensorFlow version:", tf.__version__)
print("Available GPUs:", gpus)
```

```
for gpu in gpus:
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
        print("Memory growth enabled for:", gpu)
    except RuntimeError as error:
        print(error)

if not gpus:
    print(
        "No TensorFlow GPU detected. Training will use the CPU "
        "unless the local TensorFlow/GPU configuration is fixed."
    )
```

```
TensorFlow version: 2.10.0
Available GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]
Physical devices cannot be modified after being initialized
Physical devices cannot be modified after being initialized
```

```
X_train, X_temp, y_train, y_temp = train_test_split(
    file_paths,
    labels,
    test_size=0.30,
    random_state=SEED,
    stratify=labels
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp
)

print("Training images:", len(X_train))
print("Validation images:", len(X_val))
print("Testing images:", len(X_test))
print("Total images:", len(X_train) + len(X_val) + len(X_test))
```

```
Training images: 19007
Validation images: 4073
Testing images: 4073
Total images: 27153
```

```
AUTOTUNE = tf.data.AUTOTUNE
NUM_CLASSES = len(class_names)

def load_image(path, label):
    image = tf.io.read_file(path)
    image = tf.io.decode_image(
        image, channels=3, expand_animations=False
    )
    image.set_shape([None, None, 3])
    image = tf.image.resize(image, IMAGE_SIZE)
    image = tf.cast(image, tf.float32)
```

```
    return image, tf.cast(label, tf.int32)


def make_dataset(paths, labels, training=False):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))

    if training:
        ds = ds.shuffle(
            buffer_size=min(len(paths), 10000),
            seed=42
        )

    ds = ds.map(load_image, num_parallel_calls=AUTOTUNE)
    return ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)


train_ds = make_dataset(X_train, y_train, training=True)
val_ds = make_dataset(X_val, y_val)
test_ds = make_dataset(X_test, y_test)

print("Updated image pipeline is ready.")
```

```
Updated image pipeline is ready.
```

```
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import MobileNetV2

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.10)
])

base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False

inputs = layers.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.35)(x)

outputs = layers.Dense(
    NUM_CLASSES,
    activation="softmax"
)(x)

model = Model(inputs, outputs)
```

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 224, 224, 3)]     0

 sequential (Sequential)     (None, 224, 224, 3)       0

 tf.math.truediv (TFOpLambda  (None, 224, 224, 3)      0
 )

 tf.math.subtract (TFOpLambd  (None, 224, 224, 3)      0
 a)

 mobilenetv2_1.00_224 (Funct  (None, 7, 7, 1280)       2257984
 ional)

 global_average_pooling2d (G  (None, 1280)             0
 lobalAveragePooling2D)

 dropout (Dropout)           (None, 1280)              0

 dense (Dense)               (None, 10)                12810

=================================================================
Total params: 2,270,794
Trainable params: 12,810
Non-trainable params: 2,257,984
_________________________________________________________________
```

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

classes = np.unique(np.asarray(y_train))

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=np.asarray(y_train)
)

class_weight_dict = {
    int(c): float(w)
    for c, w in zip(classes, weights)
}

print(class_weight_dict)
```

```
{0: 1.6189948892674617, 1: 1.2912364130434784, 2: 0.8647406733393994, 3: 2.1598863636363634, 4: 0.8171539122957867, 5: 0.3406882953934397, 6: 1.3063230240549828, 7: 1.32084781097984
```

```
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau,
    ModelCheckpoint
)

SAVE_DIR = EXTRACT_TO.parent

HEAD_PATH = SAVE_DIR / "best_skin_mobilenetv2_head.keras"
FINE_PATH = SAVE_DIR / "best_skin_mobilenetv2_finetuned.keras"

def get_callbacks(path):
    return [
        EarlyStopping(
            monitor="val_loss",
            patience=4,
            restore_best_weights=True
        ),
        ReduceLROnPlateau(
            monitor="val_loss",
            factor=0.5,
            patience=2,
            min_lr=1e-7
        ),
        ModelCheckpoint(
            str(path),
            monitor="val_loss",
            save_best_only=True
        )
    ]

history_head = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12,
    class_weight=class_weight_dict,
    callbacks=get_callbacks(HEAD_PATH)
)
```

```
Epoch 1/12
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
```

```
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
1188/1188 [==============================] - 286s 232ms/step - loss: 1.5163 - accuracy: 0.5052 - val_loss: 1.0081 - val_accuracy: 0.6175 - lr: 0.0010
Epoch 2/12
1188/1188 [==============================] - 276s 233ms/step - loss: 1.2862 - accuracy: 0.5769 - val_loss: 0.9369 - val_accuracy: 0.6479 - lr: 0.0010
Epoch 3/12
1188/1188 [==============================] - 268s 226ms/step - loss: 1.2483 - accuracy: 0.5898 - val_loss: 0.9899 - val_accuracy: 0.6187 - lr: 0.0010
Epoch 4/12
1188/1188 [==============================] - 266s 224ms/step - loss: 1.2172 - accuracy: 0.5944 - val_loss: 0.9715 - val_accuracy: 0.6241 - lr: 0.0010
Epoch 5/12
1188/1188 [==============================] - 269s 226ms/step - loss: 1.1569 - accuracy: 0.6127 - val_loss: 0.9170 - val_accuracy: 0.6492 - lr: 5.0000e-04
Epoch 6/12
1188/1188 [==============================] - 266s 224ms/step - loss: 1.1326 - accuracy: 0.6262 - val_loss: 0.9540 - val_accuracy: 0.6428 - lr: 5.0000e-04
Epoch 7/12
1188/1188 [==============================] - 266s 224ms/step - loss: 1.1242 - accuracy: 0.6216 - val_loss: 0.9131 - val_accuracy: 0.6514 - lr: 5.0000e-04
Epoch 8/12
1188/1188 [==============================] - 261s 220ms/step - loss: 1.1249 - accuracy: 0.6238 - val_loss: 0.9056 - val_accuracy: 0.6531 - lr: 5.0000e-04
Epoch 9/12
1188/1188 [==============================] - 280s 236ms/step - loss: 1.1172 - accuracy: 0.6192 - val_loss: 0.9909 - val_accuracy: 0.6148 - lr: 5.0000e-04
Epoch 10/12
1188/1188 [==============================] - 427s 359ms/step - loss: 1.1082 - accuracy: 0.6243 - val_loss: 0.9144 - val_accuracy: 0.6550 - lr: 5.0000e-04
Epoch 11/12
1188/1188 [==============================] - 519s 436ms/step - loss: 1.0778 - accuracy: 0.6372 - val_loss: 0.8997 - val_accuracy: 0.6521 - lr: 2.5000e-04
Epoch 12/12
1188/1188 [==============================] - 515s 433ms/step - loss: 1.0703 - accuracy: 0.6382 - val_loss: 0.9104 - val_accuracy: 0.6519 - lr: 2.5000e-04
```

```
from pathlib import Path

print("Checkpoint path:", HEAD_PATH)
print("Checkpoint exists:", Path(HEAD_PATH).exists())
```

```
Checkpoint path: C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_head.keras
Checkpoint exists: True
```

```
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau,
    ModelCheckpoint
)
from pathlib import Path

HEAD_PATH = Path(HEAD_PATH)
FINE_PATH = HEAD_PATH.parent / "best_skin_mobilenetv2_finetuned.keras"

if not HEAD_PATH.exists():
    raise FileNotFoundError(
        f"Best checkpoint not found: {HEAD_PATH}"
    )

# Load the best model from the first training phase
model = tf.keras.models.load_model(str(HEAD_PATH))
```

```
# Locate the MobileNetV2 backbone inside the model
base_model = next(
    (
        layer for layer in model.layers
        if isinstance(layer, tf.keras.Model)
        and "mobilenetv2" in layer.name.lower()
    ),
    None
)

if base_model is None:
    raise ValueError(
        "MobileNetV2 backbone not found. Check model.summary()."
    )

# Unfreeze the backbone, then freeze all but its last 30 layers
base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

# Keep batch-normalization layers frozen
for layer in base_model.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False

# Use a small learning rate for fine-tuning
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.00001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=3,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=1,
        min_lr=1e-7
    ),
    ModelCheckpoint(
        str(FINE_PATH),
        monitor="val_loss",
        save_best_only=True,
        mode="min"
    )
]

history_fine = model.fit(
```

```
    train_ds,
    validation_data=val_ds,
    epochs=8,
    class_weight=class_weight_dict,
    callbacks=callbacks
)

print("Fine-tuning completed.")
print("Best fine-tuned checkpoint:", FINE_PATH)
```

```
Epoch 1/8
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
1188/1188 [==============================] - 387s 321ms/step - loss: 1.0576 - accuracy: 0.6448 - val_loss: 0.8907 - val_accuracy: 0.6511 - lr: 1.0000e-05
Epoch 2/8
1188/1188 [==============================] - 280s 235ms/step - loss: 0.9796 - accuracy: 0.6714 - val_loss: 0.8359 - val_accuracy: 0.6823 - lr: 1.0000e-05
Epoch 3/8
1188/1188 [==============================] - 278s 234ms/step - loss: 0.9311 - accuracy: 0.6917 - val_loss: 0.8283 - val_accuracy: 0.6848 - lr: 1.0000e-05
Epoch 4/8
1188/1188 [==============================] - 278s 234ms/step - loss: 0.8883 - accuracy: 0.7112 - val_loss: 0.7905 - val_accuracy: 0.6975 - lr: 1.0000e-05
Epoch 5/8
1188/1188 [==============================] - 277s 233ms/step - loss: 0.8433 - accuracy: 0.7269 - val_loss: 0.7910 - val_accuracy: 0.6983 - lr: 1.0000e-05
Epoch 6/8
1188/1188 [==============================] - 278s 234ms/step - loss: 0.7872 - accuracy: 0.7440 - val_loss: 0.7554 - val_accuracy: 0.7147 - lr: 5.0000e-06
Epoch 7/8
1188/1188 [==============================] - 283s 238ms/step - loss: 0.7711 - accuracy: 0.7516 - val_loss: 0.7543 - val_accuracy: 0.7152 - lr: 5.0000e-06
Epoch 8/8
1188/1188 [==============================] - 279s 234ms/step - loss: 0.7499 - accuracy: 0.7582 - val_loss: 0.7405 - val_accuracy: 0.7223 - lr: 5.0000e-06
Fine-tuning completed.
Best fine-tuned checkpoint: C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
```

Start coding or generate with AI.